FMCG 4.0 Formal Verification: Complete Theorem Reference

Lean 4 / Mathlib Formalisation — 23 Numbered Machine-Checked Theorems

Alexandre Alves da Silva
UCL Computer Science

November 7, 2026

Abstract

This document presents the numbered machine-checked theorems formalising the mathematical properties of the Signal-to-Shelf (StS) metric and Coupling Benefit Ratio (CBR), as defined in the FMCG 4.0 framework, and the chain from consumer signal to commercial strategy of Appendix A of the thesis. Theorems 1 to 14 are in `FMCG40/Basic.lean` and Theorems 15 to 23 are in `FMCG40/Chain.lean`. All proofs were developed and verified in Lean 4 using the Mathlib library, with zero admitted proofs (`sorry`), zero new axioms and zero unresolved proof obligations. The complete formalisation is publicly available at: https://github.com/Alex-Alves3/FMCG40-Lean

Core Definitions

Signal-to-Shelf (StS)

$$\mathrm{StS}(R_0, \lambda, t_0, t) = \frac{R_0 \cdot e^{-\lambda t}}{t + t_0} \tag{1}$$

Market Response

$$\mathrm{MarketResponse}(R_0, \lambda, t) = R_0 \cdot e^{-\lambda t} \tag{2}$$

Temporal Decay

$$D(\lambda, t) = e^{-\lambda t} \tag{3}$$

Integrated / Sequential Intrinsic Quality

$$R_{0,\mathrm{integrated}} = R_{\mathrm{base}} + R_{\mathrm{prod}} + R_{\mathrm{mark}} + R_{\mathrm{coup}} \tag{4}$$

$$R_{0,\mathrm{sequential}} = R_{\mathrm{base}} + R_{\mathrm{prod}} + R_{\mathrm{mark}} \tag{5}$$

Coupling Benefit Ratio (CBR)

$$\mathrm{CBR} = \frac{\mathrm{StS}(R_{0,\mathrm{integrated}}, \lambda, t_0, t_{\mathrm{int}})}{\mathrm{StS}(R_{0,\mathrm{sequential}}, \lambda, t_0, t_{\mathrm{seq}})} \tag{6}$$

Coupling Contribution Percentage

$$\eta_{\mathrm{coup}} = \frac{R_{\mathrm{coup}}}{R_{0,\mathrm{integrated}}} \tag{7}$$

1 StS Denominator Positivity

Statement. For $t_0 > 0$ and $t \geq 0$, the StS denominator is strictly positive.

$$t_0 > 0,\ t \geq 0 \implies t + t_0 > 0 \tag{8}$$

```
theorem StS_denominator_pos (t0 t : R) (ht0 : t0 > 0) (
    ht : t >= 0) :
t + t0 > 0 := by
linarith
```

Significance. The foundational well-definedness fact — establishes that StS never divides by zero under the stated parameter conditions.

2 StS Well-Definedness

Statement. The StS denominator is never exactly zero.

$$t_0 > 0,\ t \geq 0 \implies t + t_0 \neq 0 \tag{9}$$

```
theorem StS_well_defined (t0 t : R) (ht0 : t0 > 0) (ht :
    t >= 0) :
t + t0 <> 0 := by
have h := StS_denominator_pos t0 t ht0 ht
linarith
```

Significance. Direct corollary of Theorem 1, stated in the form needed for division-safety arguments elsewhere.

3 Maximum StS Value at $t = 0$

Statement. At $t = 0$, StS reduces exactly to R_0/t_0.

$$\mathrm{StS}(R_0, \lambda, t_0, 0) = \frac{R_0}{t_0} \tag{10}$$

```
theorem StS_max_value (R0 lam t0 : R) (ht0 : t0 > 0) :
StS R0 lam t0 0 = R0 / t0 := by
simp [StS, Real.exp_zero]
```

Significance. Confirms the peak efficiency value the paper's prose asserts — immediate deployment yields exactly R_0/t_0, with no decay term reducing it.

4 Market Response Positivity

Statement. For $R_0 > 0$, the Market Response function is strictly positive at every t.

$$R_0 > 0 \implies R_0 \cdot e^{-\lambda t} > 0 \tag{11}$$

```
theorem MarketResponse_pos (R0 lam t : R) (hR0_pos : R0
    > 0) :
MarketResponse R0 lam t > 0 := by
unfold MarketResponse
apply mul_pos hR0_pos
exact Real.exp_pos _
```

Significance. Establishes that market response never becomes zero or negative for a strategy with positive intrinsic quality — a basic sanity property of the model.

5 Temporal Decay Bounds

Statement. For $t \geq 0$ and $\lambda > 0$, the decay function stays strictly between 0 and 1.

$$0 < e^{-\lambda t} \leq 1 \tag{12}$$

```
theorem TemporalDecay_bounds (lam t : R) (ht : t >= 0) (
    hlam : lam > 0) :
0 < TemporalDecay lam t /\ TemporalDecay lam t <= 1 :=
    by
constructor
. unfold TemporalDecay; exact Real.exp_pos _
. unfold TemporalDecay
rw [<- Real.exp_zero]
apply Real.exp_le_exp.mpr
nlinarith
```

Significance. Confirms the decay function behaves as a genuine "fraction of opportunity remaining" — never exceeding 100%, never reaching or going below 0%.

6 Trend Half-Life Identity

Statement. The formula $t_{1/2} = \ln(2)/\lambda$ correctly gives the point at which decay reaches exactly one-half.

$$D\left(\lambda, \frac{\ln 2}{\lambda}\right) = \frac{1}{2} \tag{13}$$

```
theorem trend_half_life (lam : R) (hlam : lam > 0) :
TemporalDecay lam (Real.log 2 / lam) = 1 / 2 := by
unfold TemporalDecay
have hne : lam <> 0 := ne_of_gt hlam
rw [show -lam * (Real.log 2 / lam) = -Real.log 2 by
    field_simp]
rw [Real.exp_neg, Real.exp_log (by norm_num : (2:R) > 0)
    ]
norm_num
```

Significance. Confirms the half-life formula used throughout the paper's parameter estimation discussion is exact, not approximate.

7 Uniqueness of Exponential Decay

Statement. Given $D(0) = 1$ and the derivative condition $D'(t) = -\lambda D(t)$ at every $t \geq 0$, the function D is uniquely determined to be $D(t) = e^{-\lambda t}$.

$$D(0) = 1,\ \forall t \geq 0,\ D'(t) = -\lambda D(t) \implies \forall t \geq 0,\ D(t) = e^{-\lambda t} \tag{14}$$

```
theorem decay_uniqueness_given_derivative_at_zero
(D : R -> R)
(hD0 : D 0 = 1)
(hDpos : forall t >= 0, D t > 0)
```

```
(hDfunc : forall t s, t >= 0 -> s >= 0 -> D (t + s) = D
   t * D s)
(lam : R)
(hDderiv_all : forall t >= 0, HasDerivAt D (-lam * D t)
   t) :
forall t >= 0, D t = Real.exp (-lam * t) := by
intro t ht
set E : R -> R := fun s => D s * Real.exp (lam * s) with
    hE_def
have hEderiv_pt : forall s >= 0, HasDerivAt E 0 s := by
intro s hs0
have hs1 : HasDerivAt (fun u : R => Real.exp (lam * u))
   (Real.exp (lam * s) * lam) s := by
have hlin : HasDerivAt (fun u : R => lam * u) lam s :=
   by
simpa using (hasDerivAt_id s).const_mul lam
simpa [mul_comm] using hlin.exp
have h2 := (hDderiv_all s hs0).mul hs1
have heq : -lam * D s * Real.exp (lam * s) + D s * (Real
   .exp (lam * s) * lam) = 0 := by
ring
rw [<- heq]
exact h2
have hEderiv_within : forall s in Set.Ico (0:R) t,
   HasDerivWithinAt E 0 (Set.Ici s) s :=
fun s hs => (hEderiv_pt s hs.1).hasDerivWithinAt
have hEcont : ContinuousOn E (Set.Icc 0 t) :=
HasDerivAt.continuousOn (fun x hx => hEderiv_pt x hx.1)
have hbound : forall x in Set.Ico (0:R) t, ||(0:R)|| <=
   (0:R) := by
intro x _; simp
have hkey :=
   norm_image_sub_le_of_norm_deriv_right_le_segment
(f := E) (f' := fun _ => (0:R)) (a := (0:R)) (b := t) (C
    := (0:R))
hEcont hEderiv_within hbound
have hEt : E t = E 0 := by
have h0 : ||E t - E 0|| <= 0 * (t - 0) := hkey t (Set.
   right_mem_Icc.mpr ht)
have h0' : E t - E 0 = 0 := by
have h1 : ||E t - E 0|| <= 0 := by linarith
exact norm_le_zero_iff.mp h1
linarith [h0']
have hE0 : E 0 = 1 := by simp [hE_def, hD0]
have hEt1 : E t = 1 := by rw [hEt, hE0]
have hkey2 : D t * Real.exp (lam * t) = 1 := hEt1
have hExp_pos : Real.exp (lam * t) > 0 := Real.exp_pos _
have htarget : D t * Real.exp (lam * t) = Real.exp (-lam
    * t) * Real.exp (lam * t) := by
rw [hkey2]
rw [<- Real.exp_add]
have : -lam * t + lam * t = 0 := by ring
rw [this, Real.exp_zero]
exact mul_right_cancel0 (ne_of_gt hExp_pos) htarget
```

Significance. Closes the gap between the paper's prose claim ("the unique continuous function satisfying the memoryless property") and formal verification. This proof establishes uniqueness under an *explicit differentiability hypothesis* — a genuinely honest, if narrower, claim than the

fully general classical theorem (which derives differentiability from continuity plus the functional equation alone, and remains unformalised here). Proof strategy: construct the auxiliary function $E(s) = D(s)e^{\lambda s}$, show it has zero derivative everywhere on $[0, t]$, invoke a mean-value-theorem-based constancy result to conclude E is constant on the interval, then solve algebraically for $D(t)$.

8 Coupling Contribution Bound

Statement. Given $R_{\text{coup}} \geq 0$ and $R_{\text{base}} + R_{\text{prod}} + R_{\text{mark}} > 0$, the coupling contribution percentage lies strictly in $[0, 1)$.

$$0 \leq \eta_{\text{coup}} < 1 \tag{15}$$

```
theorem eta_coup_bounds
(R_base R_prod R_mark R_coup : R)
(hR_coup : R_coup >= 0)
(hR_base_pos : R_base + R_prod + R_mark > 0) :
0 <= eta_coup R_base R_prod R_mark R_coup /\
eta_coup R_base R_prod R_mark R_coup < 1 := by
have hden_pos : R0_integrated R_base R_prod R_mark
   R_coup > 0 := by
unfold R0_integrated
linarith
unfold eta_coup
constructor
. exact div_nonneg hR_coup (le_of_lt hden_pos)
. rw [div_lt_one hden_pos]
unfold R0_integrated
linarith
```

Significance. Confirms η_{coup} behaves as a valid percentage — it can never exceed 100% of the total strategy quality, and is never negative.

9 Decay Rate Estimator Positivity

Statement. Given two trend strength measurements $TS_1 > TS_2 > 0$ at times $t_1 < t_2$, the estimated decay rate is positive.

$$TS_1 > TS_2 > 0,\ t_1 < t_2 \implies \frac{\ln(TS_1/TS_2)}{t_2 - t_1} > 0 \tag{16}$$

```
theorem decay_rate_estimator_well_defined
(TS1 TS2 t1 t2 : R)
(hTS1 : TS1 > 0) (hTS2 : TS2 > 0)
(hTS_decay : TS1 > TS2)
(ht_order : t1 < t2) :
Real.log (TS1 / TS2) / (t2 - t1) > 0 := by
apply div_pos
. apply Real.log_pos
rw [lt_div_iff0 hTS2]
linarith
. linarith
```

Significance. Validates the parameter estimation procedure described in the paper — a decaying trend (measured strength decreasing over time) always produces a sensible, positive λ estimate.

10 Strict Monotonicity of StS

Statement. For $R_0 > 0$, $\lambda > 0$, $t_0 > 0$, the StS function is strictly decreasing on $[0, \infty)$.

$$\mathrm{StS}(R_0, \lambda, t_0, \cdot) \text{ is strictly decreasing on } [0, \infty) \tag{17}$$

```
theorem StS_strictly_decreasing
(R0 lam t0 : R) (hR0_pos : R0 > 0) (hlam : lam > 0) (ht0
    : t0 > 0) :
StrictAntiOn (StS R0 lam t0) (Ici 0) := by
have hderiv : forall t in Ici (0:R),
HasDerivAt (StS R0 lam t0)
(R0 * (Real.exp (-lam * t) * (-lam * (t + t0) - 1)) / (t
    + t0)^2) t := by
intro t ht
have ht' : t >= 0 := ht
have hnum : HasDerivAt (fun s : R => R0 * Real.exp (-lam
    * s))
(R0 * (Real.exp (-lam * t) * (-lam))) t := by
have h1 : HasDerivAt (fun s : R => -lam * s) (-lam) t :=
    by
simpa using (hasDerivAt_id t).const_mul (-lam)
have h2 : HasDerivAt (fun s : R => Real.exp (-lam * s))
(Real.exp (-lam * t) * (-lam)) t := h1.exp
simpa [mul_assoc] using h2.const_mul R0
have hden : HasDerivAt (fun s : R => s + t0) 1 t := by
simpa using (hasDerivAt_id t).add_const t0
have hdenne : (t + t0) <> 0 := ne_of_gt (
   StS_denominator_pos t0 t ht0 ht')
have hcomb := hnum.div hden hdenne
have hfun : StS R0 lam t0 = (fun s : R => R0 * Real.exp
   (-lam * s)) / (fun s : R => s + t0) := by
funext s
simp [StS, Pi.div_apply]
rw [hfun]
have heq : R0 * (Real.exp (-lam * t) * (-lam * (t + t0)
   - 1)) / (t + t0) ^ 2
= (R0 * (Real.exp (-lam * t) * (-lam)) * (t + t0) - R0 *
    Real.exp (-lam * t) * 1)
/ (t + t0) ^ 2 := by ring
rw [heq]
exact hcomb
apply strictAntiOn_of_deriv_neg (convex_Ici (0:R))
. apply ContinuousOn.div
. exact (continuous_const.mul (Real.continuous_exp.comp
(continuous_const.mul continuous_id))).continuousOn
. exact (continuous_id.add continuous_const).
   continuousOn
. intro t ht
exact ne_of_gt (StS_denominator_pos t0 t ht0 ht)
. intro t ht
rw [interior_Ici] at ht
have ht' : t >= 0 := le_of_lt ht
have hd := (hderiv t ht').deriv
rw [hd]
have hexp_pos : Real.exp (-lam * t) > 0 := Real.exp_pos
   _
have hden_pos : (t + t0)^2 > 0 := by positivity
```

```
    have hfactor_neg : (-lam * (t + t0) - 1) < 0 := by
    have hpos : lam * (t + t0) > 0 := mul_pos hlam (by
        linarith)
    linarith
    have hnum_neg : R0 * (Real.exp (-lam * t) * (-lam * (t +
         t0) - 1)) < 0 := by
    apply mul_neg_of_pos_of_neg hR0_pos
    exact mul_neg_of_pos_of_neg hexp_pos hfactor_neg
    exact div_neg_of_neg_of_pos hnum_neg hden_pos
```

Significance. The formal justification for "acting faster is always better" — proved via direct computation of the derivative and showing it is negative everywhere. This is the mathematical backbone of the paper's central speed argument.

11 Maximum StS at $t = 0$ (Global)

Statement. The maximum value of StS over all $t \geq 0$ occurs at $t = 0$.

$$\forall t \geq 0,\ \mathrm{StS}(R_0, \lambda, t_0, t) \leq \mathrm{StS}(R_0, \lambda, t_0, 0) \tag{18}$$

```
    theorem StS_max_at_zero
    (R0 lam t0 t : R)
    (hR0_pos : R0 > 0) (hlam : lam > 0) (ht0 : t0 > 0) (ht :
         t >= 0) :
    StS R0 lam t0 t <= StS R0 lam t0 0 := by
    have hanti : AntitoneOn (StS R0 lam t0) (Ici 0) :=
    (StS_strictly_decreasing R0 lam t0 hR0_pos hlam ht0).
        antitoneOn
    exact hanti (mem_Ici.mpr (le_refl 0)) (mem_Ici.mpr ht)
        ht
```

Significance. Direct consequence of Theorem 10 — confirms the maximum achievable efficiency R_0/t_0 (Theorem 3) is genuinely the global maximum, not merely a local one.

12 Sufficient Conditions for CBR ≥ 1

Statement. Given $t_{\text{int}} \geq 0$, $t_{\text{seq}} \geq 0$, $t_{\text{int}} \leq t_{\text{seq}}$, $R_{\text{coup}} > 0$, and $R_{\text{base}} + R_{\text{prod}} + R_{\text{mark}} > 0$, the Coupling Benefit Ratio is at least 1.

$$\mathrm{CBR} \geq 1 \tag{19}$$

```
    theorem CBR_geq_one
    (R_base R_prod R_mark R_coup lam t0 t_int t_seq : R)
    (hlam : lam > 0) (ht0 : t0 > 0)
    (ht_int : t_int >= 0)
    (ht_seq : t_seq >= 0)
    (ht : t_int <= t_seq)
    (hcoup : R_coup > 0)
    (hR_base_pos : R_base + R_prod + R_mark > 0) :
    CBR R_base R_prod R_mark R_coup lam t0 t_int t_seq >= 1
        := by
    set R0int := R_base + R_prod + R_mark + R_coup with
        hR0int_def
    set R0seq := R_base + R_prod + R_mark with hR0seq_def
    have hR0int_pos : R0int > 0 := by rw [hR0int_def];
        linarith
```

```
    have hR0seq_pos : R0seq > 0 := by rw [hR0seq_def];
       linarith
    have hR0_lt : R0seq < R0int := by rw [hR0int_def,
       hR0seq_def]; linarith
    have hden_int_pos : t_int + t0 > 0 :=
       StS_denominator_pos t0 t_int ht0 ht_int
    have hden_seq_pos : t_seq + t0 > 0 :=
       StS_denominator_pos t0 t_seq ht0 ht_seq
    have hStS_seq_pos : StS R0seq lam t0 t_seq > 0 := by
    unfold StS
    exact div_pos (mul_pos hR0seq_pos (Real.exp_pos _))
       hden_seq_pos
    have step_a : StS R0seq lam t0 t_seq <= StS R0int lam t0
        t_seq := by
    unfold StS
    apply div_le_div_of_nonneg_right _ (le_of_lt
       hden_seq_pos)
    exact mul_le_mul_of_nonneg_right (le_of_lt hR0_lt) (
       le_of_lt (Real.exp_pos _))
    have step_b : StS R0int lam t0 t_seq <= StS R0int lam t0
        t_int := by
    rcases eq_or_lt_of_le ht with heq | hlt
    . rw [heq]
    . have hanti : AntitoneOn (StS R0int lam t0) (Ici 0) :=
    (StS_strictly_decreasing R0int lam t0 hR0int_pos hlam
       ht0).antitoneOn
    exact hanti (mem_Ici.mpr ht_int) (mem_Ici.mpr ht_seq) ht
    unfold CBR
    rw [ge_iff_le, le_div_iff0 hStS_seq_pos]
    calc 1 * StS R0seq lam t0 t_seq = StS R0seq lam t0 t_seq
        := by ring
    _ <= StS R0int lam t0 t_seq := step_a
    _ <= StS R0int lam t0 t_int := step_b
```

Significance. The paper's central claim, now genuinely proved rather than merely asserted — when integration is at least as fast as the sequential alternative AND provides positive coupling value, integration is mathematically guaranteed to be at least as efficient. Proof combines Theorem 10 (monotonicity in time) with a separate monotonicity-in-R_0 argument.

13 CBR Invariance to λ (Equal Deployment Times)

Statement. When $t_{\text{int}} = t_{\text{seq}} = t$, the Coupling Benefit Ratio is exactly invariant to the choice of λ.

$$t_{\text{int}} = t_{\text{seq}} \implies \text{CBR}(\lambda_1) = \text{CBR}(\lambda_2)\ \forall \lambda_1, \lambda_2 > 0 \tag{20}$$

```
    theorem CBR_lambda_invariant_equal_time
    (R_base R_prod R_mark R_coup t0 t lam1 lam2 : R)
    (ht0 : t0 > 0) (ht : t >= 0)
    (hlam1 : lam1 > 0) (hlam2 : lam2 > 0)
    (hR_base_pos : R_base + R_prod + R_mark > 0)
    (hcoup : R_coup > 0) :
    CBR R_base R_prod R_mark R_coup lam1 t0 t t =
    CBR R_base R_prod R_mark R_coup lam2 t0 t t := by
    have hden_pos : t + t0 > 0 := StS_denominator_pos t0 t
       ht0 ht
    have hden_ne : t + t0 <> 0 := ne_of_gt hden_pos
```

```
    have hR0seq_pos : R_base + R_prod + R_mark > 0 :=
       hR_base_pos
    have hseq_ne : R_base + R_prod + R_mark <> 0 := ne_of_gt
        hR0seq_pos
    unfold CBR StS
    have key : forall lam' : R,
    (R_base + R_prod + R_mark + R_coup) * Real.exp (-lam' *
       t) / (t + t0) /
    ((R_base + R_prod + R_mark) * Real.exp (-lam' * t) / (t
       + t0))
    = (R_base + R_prod + R_mark + R_coup) / (R_base + R_prod
        + R_mark) := by
    intro lam'
    have hexp_ne : Real.exp (-lam' * t) <> 0 := ne_of_gt (
       Real.exp_pos _)
    field_simp
    rw [key lam1, key lam2]
```

Significance. Formal basis for the paper's sensitivity-analysis discussion — in the equal-deployment-time special case, the exponential decay terms cancel algebraically, leaving CBR dependent only on the ratio of intrinsic quality terms. Note: this result does *not* extend to the general unequal-time case (explicitly flagged as an open limitation in the paper).

14 CBR Equals 1 Under No Coupling

Statement. When $R_{\text{coup}} = 0$ and $t_{\text{int}} = t_{\text{seq}}$, the Coupling Benefit Ratio equals exactly 1.

$$R_{\text{coup}} = 0,\ t_{\text{int}} = t_{\text{seq}} \implies \text{CBR} = 1 \tag{21}$$

```
    theorem CBR_eq_one_iff_no_coupling
    (R_base R_prod R_mark lam t0 t : R)
    (ht0 : t0 > 0) (ht : t >= 0)
    (hR_base_pos : R_base + R_prod + R_mark > 0) :
    CBR R_base R_prod R_mark 0 lam t0 t t = 1 := by
    have hden : t + t0 > 0 := StS_denominator_pos t0 t ht0
       ht
    have hStS_pos : StS (R_base + R_prod + R_mark) lam t0 t
       > 0 := by
    unfold StS
    apply div_pos
    . exact mul_pos hR_base_pos (Real.exp_pos _)
    . exact hden
    unfold CBR
    rw [add_zero]
    exact div_self (ne_of_gt hStS_pos)
```

Significance. Confirms the paper's stated interpretation that CBR = 1 indicates "no integration advantage" — precisely in the case where there is no coupling value and both deployment paths take equal time, integration provides mathematically zero benefit, exactly as the model predicts.

15 Decomposition of $\ln \mathrm{CBR}$ (Equation A.33)

Statement. For $R > 0$, $p > 0$, $\lambda \in \mathbb{R}$ and $\Delta t_{\mathrm{int}}, \Delta t_{\mathrm{seq}} \in \mathbb{R}$, with $\mathrm{CBR} = \frac{R}{p} \exp\big(-\lambda(\Delta t_{\mathrm{int}} - \Delta t_{\mathrm{seq}})\big)$ (Equation A.32),

$$\ln \mathrm{CBR} = \ln \frac{R}{p} + \lambda\,(\Delta t_{\mathrm{seq}} - \Delta t_{\mathrm{int}}). \tag{22}$$

```
/-- Equation A.32: CBR computed by the implementation, with
    R = R(a*_B; S) and p the portfolio coverage. The times are
    Dt_int and Dt_seq, in the same unit as 1/lam. -/
noncomputable def CBR_impl (R p lam dt_int dt_seq : R) : R :=
  R / p * Real.exp (-lam * (dt_int - dt_seq))

theorem CBR_log_decomposition
    (R p lam dt_int dt_seq : R) (hR : R > 0) (hp : p > 0) :
    Real.log (CBR_impl R p lam dt_int dt_seq) =
    Real.log (R / p) + lam * (dt_seq - dt_int) := by
  unfold CBR_impl
  rw [Real.log_mul (div_pos hR hp).ne' (Real.exp_pos _).ne', Real.
     log_exp]
  ring
```

Significance. The first term is the quality gain of the recipe over the portfolio product; the second is the speed gain. The decomposition holds for every real λ and every pair of times.

16 Exact Condition for Coupling Benefit (Section A.6)

Statement. With the hypotheses of Theorem 15,

$$\mathrm{CBR} > 1 \iff \frac{R}{p} > \exp\big(-\lambda(\Delta t_{\mathrm{seq}} - \Delta t_{\mathrm{int}})\big). \tag{23}$$

Relation to condition 10 of Section A.1. Equation A.10 (with the denominator $t + t_0$) and Equation A.32 differ by the factor $(\Delta t_{\mathrm{seq}}+t_0)/(\Delta t_{\mathrm{int}}+t_0)$ of Equation A.34 (`CBR_arms_eq_CBR_StS`). Under the hypotheses of condition 10 ($\lambda > 0$, $t_0 > 0$, $0 \le t_{\mathrm{int}} \le t_{\mathrm{seq}}$, $R_{\mathrm{coup}} > 0$, $R_{\mathrm{base}} + R_{\mathrm{prod}} + R_{\mathrm{mark}} > 0$), with $R = R_{\mathrm{base}} + R_{\mathrm{prod}} + R_{\mathrm{mark}} + R_{\mathrm{coup}}$ and $p = R_{\mathrm{base}} + R_{\mathrm{prod}} + R_{\mathrm{mark}}$, both Equation A.32 and Equation A.10 are strictly greater than 1 (`CBR_condition10_strict`).

```
theorem CBR_gt_one_iff
    (R p lam dt_int dt_seq : R) (_hR : R > 0) (_hp : p > 0) :
    CBR_impl R p lam dt_int dt_seq > 1 <->
    R / p > Real.exp (-lam * (dt_seq - dt_int)) := by
  have hE : 0 < Real.exp (-lam * (dt_seq - dt_int)) := Real.exp_pos _
  have h : Real.exp (-lam * (dt_int - dt_seq)) =
      (Real.exp (-lam * (dt_seq - dt_int)))^-1 := by
    rw [<- Real.exp_neg]
    congr 1
    ring
  unfold CBR_impl
  rw [h, <- div_eq_mul_inv]
  exact one_lt_div hE

/-- Equation A.34: the full Signal-to-Shelf form, with the factor
    (Dt_seq + t0) / (Dt_int + t0). -/
noncomputable def CBR_StS (R p lam t0 dt_int dt_seq : R) : R :=
  CBR_impl R p lam dt_int dt_seq * ((dt_seq + t0) / (dt_int + t0))
```

```
/-- Equation A.10 with arbitrary intrinsic qualities for the two arms.
    `CBR R_base R_prod R_mark R_coup ...` of Basic.lean is the instance
    with R0_int = R_base + R_prod + R_mark + R_coup and
    R0_seq = R_base + R_prod + R_mark. -/
noncomputable def CBR_arms (R0_int R0_seq lam t0 t_int t_seq : R) : R :=
  StS R0_int lam t0 t_int / StS R0_seq lam t0 t_seq

theorem CBR_eq_CBR_arms
    (R_base R_prod R_mark R_coup lam t0 t_int t_seq : R) :
    CBR R_base R_prod R_mark R_coup lam t0 t_int t_seq =
    CBR_arms (R_base + R_prod + R_mark + R_coup) (R_base + R_prod +
       R_mark)
      lam t0 t_int t_seq := rfl

/-- Relation between Equations A.32, A.34 and A.10. -/
theorem CBR_arms_eq_CBR_StS
    (R p lam t0 t_int t_seq : R)
    (hR : R > 0) (hp : p > 0)
    (ht0 : t0 > 0) (ht_int : t_int >= 0) (ht_seq : t_seq >= 0) :
    CBR_arms R p lam t0 t_int t_seq = CBR_StS R p lam t0 t_int t_seq :=
       by
  have hi : t_int + t0 > 0 := StS_denominator_pos t0 t_int ht0 ht_int
  have hs : t_seq + t0 > 0 := StS_denominator_pos t0 t_seq ht0 ht_seq
  have e : Real.exp (-lam * t_int) =
      Real.exp (-lam * (t_int - t_seq)) * Real.exp (-lam * t_seq) := by
    rw [<- Real.exp_add]
    congr 1
    ring
  unfold CBR_arms CBR_StS CBR_impl StS
  rw [e]
  have h1 : Real.exp (-lam * t_seq) <> 0 := (Real.exp_pos _).ne'
  field_simp

/-- Condition 10 of Section A.1 as a special case of the exact
    condition: under its hypotheses the criterion of
    `CBR_gt_one_iff` holds, and the conclusion is strict. -/
theorem CBR_condition10_strict
    (R_base R_prod R_mark R_coup lam t0 t_int t_seq : R)
    (hlam : lam > 0) (ht0 : t0 > 0)
    (ht_int : t_int >= 0) (ht_seq : t_seq >= 0) (ht : t_int <= t_seq)
    (hcoup : R_coup > 0)
    (hR_base_pos : R_base + R_prod + R_mark > 0) :
    CBR_impl (R_base + R_prod + R_mark + R_coup) (R_base + R_prod +
       R_mark)
      lam t_int t_seq > 1 /\
    CBR R_base R_prod R_mark R_coup lam t0 t_int t_seq > 1 := by
  have hp : R_base + R_prod + R_mark > 0 := hR_base_pos
  have hR : R_base + R_prod + R_mark + R_coup > 0 := by linarith
  have hratio : (R_base + R_prod + R_mark + R_coup) / (R_base + R_prod +
      R_mark) > 1 := by
    rw [gt_iff_lt, one_lt_div hp]
    linarith
  have hexp : Real.exp (-lam * (t_seq - t_int)) <= 1 := by
    rw [<- Real.exp_zero]
    apply Real.exp_le_exp.mpr
    nlinarith
  have h1 : CBR_impl (R_base + R_prod + R_mark + R_coup) (R_base +
     R_prod + R_mark)
```

```
      lam t_int t_seq > 1 :=
    (CBR_gt_one_iff _ _ lam t_int t_seq hR hp).mpr (lt_of_le_of_lt hexp
       hratio)
  refine <<h1, ?_>>
  rw [CBR_eq_CBR_arms,
    CBR_arms_eq_CBR_StS _ _ lam t0 t_int t_seq hR hp ht0 ht_int ht_seq]
  unfold CBR_StS
  have hi : t_int + t0 > 0 := StS_denominator_pos t0 t_int ht0 ht_int
  have hfac : (t_seq + t0) / (t_int + t0) >= 1 := by
    rw [ge_iff_le, le_div_iff0 hi]
    linarith
  calc (1 : R) < CBR_impl (R_base + R_prod + R_mark + R_coup) (R_base +
     R_prod + R_mark)
        lam t_int t_seq := h1
    _ = CBR_impl (R_base + R_prod + R_mark + R_coup) (R_base + R_prod +
       R_mark)
        lam t_int t_seq * 1 := by ring
    _ <= CBR_impl (R_base + R_prod + R_mark + R_coup) (R_base + R_prod +
       R_mark)
        lam t_int t_seq * ((t_seq + t0) / (t_int + t0)) :=
      mul_le_mul_of_nonneg_left hfac (by linarith)
```

Significance. Condition 10 is a special case in the sense that its hypotheses imply the criterion above. Its conclusion CBR ≥ 1 is weaker than what holds: the inequality is strict. The criterion itself is stated for Equation A.32, which has no t_0; for Equation A.10 the same bound carries the extra factor of Equation A.34.

17 Boundedness of the Rewards (Section A.6)

Statement. (a) For finite non-empty E and S, if every $c(\varphi, k) = \cos(\varphi(d), \varphi(e_k)) \in [-1, 1]$, then $R = \frac{1}{|E|} \sum_{\varphi \in E} \min_{k \in S} c(\varphi, k) \in [-1, 1]$ (Equation A.22).

(b) For finite non-empty S, if $\hat{y}_k = \pi_k \cdot \mathrm{expit}(f_k)$ with $\pi_k \in [0, 1]$, then $R_c = \min_{k \in S} \hat{y}_k \in [0, 1]$ (Equation A.35).

```
/-- Equation A.22. `enc` ranges over the finite encoder set E, `cos` is
    an abstract similarity, `d` stands for desc(a) and `e k` for the
    descriptor text of signal member k. -/
noncomputable def coverageReward {T V iota : Type*}
    (cos : V -> V -> R) (E : Finset (T -> V)) (S : Finset iota) (hS : S.
       Nonempty)
    (d : T) (e : iota -> T) : R :=
  (1 / (E.card : R)) *
    sum phi in E, S.inf' hS (fun k => cos (phi d) (phi (e k)))

/-- T17(a). Equation A.22 lies in [-1, 1] when cos takes values in [-1,
   1]. -/
theorem coverageReward_bounds {T V iota : Type*}
    (cos : V -> V -> R) (hcos : forall u v, -1 <= cos u v /\ cos u v <=
       1)
    (E : Finset (T -> V)) (S : Finset iota) (hE : E.Nonempty) (hS : S.
       Nonempty)
    (d : T) (e : iota -> T) :
    -1 <= coverageReward cos E S hS d e /\ coverageReward cos E S hS d e
        <= 1 := by
  have hcard : (0 : R) < (E.card : R) := by
    exact_mod_cast Finset.card_pos.mpr hE
```

```
  have hmin : forall phi in E, -1 <= S.inf' hS (fun k => cos (phi d) (
     phi (e k))) /\
      S.inf' hS (fun k => cos (phi d) (phi (e k))) <= 1 := by
    intro phi _
    constructor
    . exact (Finset.le_inf' hS _ (fun k _ => (hcos _ _).1))
    . obtain <<k, hk>> := hS
      exact (Finset.inf'_le _ hk).trans (hcos _ _).2
  have hsum_le : sum phi in E, S.inf' hS (fun k => cos (phi d) (phi (e k
     ))) <= (E.card : R) := by
    calc sum phi in E, S.inf' hS (fun k => cos (phi d) (phi (e k))) <=
       sum _phi in E, (1 : R) :=
          Finset.sum_le_sum (fun phi hphi => (hmin phi hphi).2)
      _ = (E.card : R) := by simp
  have hsum_ge : -(E.card : R) <= sum phi in E, S.inf' hS (fun k => cos
     (phi d) (phi (e k))) := by
    calc -(E.card : R) = sum _phi in E, (-1 : R) := by simp
      _ <= sum phi in E, S.inf' hS (fun k => cos (phi d) (phi (e k))) :=
          Finset.sum_le_sum (fun phi hphi => (hmin phi hphi).1)
  unfold coverageReward
  constructor
  . rw [one_div, inv_mul_eq_div, le_div_iff0 hcard]
    linarith
  . rw [one_div, inv_mul_eq_div, div_le_one hcard]
    exact hsum_le

/-- The logistic function of Equation A.35. -/
noncomputable def expit (x : R) : R := 1 / (1 + Real.exp (-x))

theorem expit_mem_Ioo (x : R) : 0 < expit x /\ expit x < 1 := by
  have h : 0 < 1 + Real.exp (-x) := by positivity
  constructor
  . unfold expit; positivity
  . unfold expit
    rw [div_lt_one h]
    linarith [Real.exp_pos (-x)]

/-- Equation A.35. `pk` and `f` are the two parts of the gradient
   boosting
    model for signal member k, evaluated at the recipe. -/
noncomputable def consumerReward {iota : Type*}
    (S : Finset iota) (hS : S.Nonempty) (pk f : iota -> R) : R :=
  S.inf' hS (fun k => pk k * expit (f k))

/-- T17(b). Equation A.35 lies in [0, 1] when each pk k is in [0, 1]. -/
theorem consumerReward_bounds {iota : Type*}
    (S : Finset iota) (hS : S.Nonempty) (pk f : iota -> R)
    (hpk : forall k, 0 <= pk k /\ pk k <= 1) :
    0 <= consumerReward S hS pk f /\ consumerReward S hS pk f <= 1 := by
  have hy : forall k, 0 <= pk k * expit (f k) /\ pk k * expit (f k) <= 1
     := by
    intro k
    obtain <<h0, h1>> := expit_mem_Ioo (f k)
    obtain <<hp0, hp1>> := hpk k
    constructor
    . positivity
    . calc pk k * expit (f k) <= 1 * 1 :=
          mul_le_mul hp1 h1.le h0.le zero_le_one
```

```
        _ = 1 := by ring
  unfold consumerReward
  constructor
  . exact Finset.le_inf' hS _ (fun k _ => (hy k).1)
  . obtain <<k, hk>> := hS
    exact (Finset.inf'_le _ hk).trans (hy k).2
```

Significance. Encoders and the cosine are abstract: the theorem holds for any encoder set and any similarity with values in $[-1, 1]$. Non-emptiness of E and S is a hypothesis.

18 Budget Monotonicity (Section A.6, Equation A.23)

Statement. For a sequence of proposals $a_1, a_2, \ldots$ and a real reward R, $M(B) = \max_{n \le B} R(a_n)$ is non-decreasing in $B \ge 1$.

```
/-- M(B) = max over n <= B of R(a_n), for a sequence of proposals
    a_1, a_2, ... and B >= 1. -/
noncomputable def bestWithin {alpha : Type*} (R : alpha -> R) (a : N ->
   alpha)
    (B : N) (hB : 1 <= B) : R :=
  (Finset.Icc 1 B).sup' <<1, Finset.mem_Icc.mpr <<le_refl 1, hB>>>> (fun
      n => R (a n))

theorem bestWithin_mono {alpha : Type*} (R : alpha -> R) (a : N -> alpha
   )
    {B1 B2 : N} (h1 : 1 <= B1) (h : B1 <= B2) :
    bestWithin R a B1 h1 <= bestWithin R a B2 (h1.trans h) := by
  unfold bestWithin
  apply Finset.sup'_mono
  intro n hn
  rw [Finset.mem_Icc] at hn |-
  exact <<hn.1, hn.2.trans h>>
```

Significance. The sequence is fixed and independent of B, so $a_1, \ldots, a_B$ is a prefix of $a_1, \ldots, a_{B'}$ for $B \le B'$. A search method whose proposals depend on the declared budget is outside this statement.

19 Safety of the Strategy Operator (Section A.6, Equations A.27 and A.28)

Statement. If $\Omega_{20} \cap \mathcal{C} \neq \emptyset$ and ω^* is an argmax of f over $\Omega_{20} \cap \mathcal{C}$ (finite sets), then $\omega^* \in \mathcal{C}$. If the intersection is empty, the operator returns no strategy. The verifier V is an abstract function with values SAT or UNSAT.

Relation to Theorem C.1. In Equation A.27, $\mathcal{C}(x)$ is defined as the set that V accepts, so the membership $\omega^* \in \mathcal{C}(x)$ follows from the definition of an argmax taken over a subset of $\mathcal{C}(x)$. The set C of Theorem C.1 is the set of constraint-compliant strategies. The last theorem of this section shows that, if V is sound with respect to C, the selected strategy lies in C (safety), and, if V is also complete and Ω_{20} meets C, a strategy is returned (relative completeness).

```
/-- Verdict of the ASP verifier. -/
inductive Verdict where
  | SAT
  | UNSAT
  deriving DecidableEq
```

```
/-- Equation A.27: the set accepted by the verifier. -/
def compliantSet {X Om : Type*} [Fintype Om] (V : X -> Om -> Verdict) (x
    : X) : Finset Om :=
  Finset.univ.filter (fun om => V x om = Verdict.SAT)

theorem mem_compliantSet {X Om : Type*} [Fintype Om] (V : X -> Om ->
   Verdict) (x : X)
    (om : Om) : om in compliantSet V x <-> V x om = Verdict.SAT := by
  simp [compliantSet]

/-- om is an argmax of f over the finite set s. -/
def IsArgmaxOn {Om : Type*} (f : Om -> R) (s : Finset Om) (om : Om) :
   Prop :=
  om in s /\ forall om' in s, f om' <= f om

/-- T19. Safety: an argmax of f over Om20 cap Cal belongs to Cal. In
    this
    encoding the statement follows from the definition of argmax over a
    set contained in Cal, given Equation A.27. -/
theorem selection_mem_compliant {X Om : Type*} [Fintype Om] [DecidableEq
    Om]
    (V : X -> Om -> Verdict) (x : X) (Om20 : Finset Om) (f : Om -> R) (
       omstar : Om)
    (_hne : (Om20 cap compliantSet V x).Nonempty)
    (harg : IsArgmaxOn f (Om20 cap compliantSet V x) omstar) :
    omstar in compliantSet V x /\ V x omstar = Verdict.SAT := by
  have hmem := (Finset.mem_inter.mp harg.1).2
  exact <<hmem, (mem_compliantSet V x omstar).mp hmem>>

/-- Equation A.28 as a partial function: the selected strategy, or
    `none` when Om20 cap Cal is empty. -/
noncomputable def selectStrategy {X Om : Type*} [Fintype Om] [
   DecidableEq Om]
    (V : X -> Om -> Verdict) (x : X) (Om20 : Finset Om) (f : Om -> R) :
       Option Om :=
  if h : (Om20 cap compliantSet V x).Nonempty then
    some (Finset.exists_max_image _ f h).choose
  else none

theorem selectStrategy_eq_none_iff {X Om : Type*} [Fintype Om] [
   DecidableEq Om]
    (V : X -> Om -> Verdict) (x : X) (Om20 : Finset Om) (f : Om -> R) :
    selectStrategy V x Om20 f = none <-> Om20 cap compliantSet V x = {}
       := by
  unfold selectStrategy
  by_cases h : (Om20 cap compliantSet V x).Nonempty
  . rw [dif_pos h]
    simp only [reduceCtorEq, false_iff]
    exact Finset.nonempty_iff_ne_empty.mp h
  . rw [dif_neg h]
    simp only [true_iff]
    exact Finset.not_nonempty_iff_eq_empty.mp h

/-- T19 for the partial function: whatever Gamma returns is an argmax of
    f
    over Om20 cap Cal, hence verified by V. -/
theorem selectStrategy_safe {X Om : Type*} [Fintype Om] [DecidableEq Om]
```

```
    (V : X -> Om -> Verdict) (x : X) (Om20 : Finset Om) (f : Om -> R) (
       om : Om)
    (h : selectStrategy V x Om20 f = some om) :
    IsArgmaxOn f (Om20 cap compliantSet V x) om /\ V x om = Verdict.SAT
       := by
  unfold selectStrategy at h
  by_cases hne : (Om20 cap compliantSet V x).Nonempty
  . rw [dif_pos hne] at h
    have hc := (Finset.exists_max_image _ f hne).choose_spec
    have h' := Option.some.inj h
    rw [h'] at hc
    exact <<<<hc.1, hc.2>>, (mem_compliantSet V x _).mp (Finset.
       mem_inter.mp hc.1).2>>
  . rw [dif_neg hne] at h
    exact absurd h (by simp)

/-- Link with Theorem C.1. If V is sound with respect to the semantic
    compliant set C, the selected strategy lies in C (safety). If V is
    also complete and Om20 meets C, a strategy is returned (relative
    completeness). -/
theorem selection_theorem_C1 {X Om : Type*} [Fintype Om] [DecidableEq Om
   ]
    (V : X -> Om -> Verdict) (x : X) (C : Set Om) (Om20 : Finset Om) (f
       : Om -> R)
    (hsound : forall om, V x om = Verdict.SAT -> om in C)
    (hcomplete : forall om, om in C -> V x om = Verdict.SAT) :
    (forall om, selectStrategy V x Om20 f = some om -> om in C) /\
    ((exists om in Om20, om in C) -> selectStrategy V x Om20 f <> none)
       := by
  constructor
  . intro om h
    exact hsound om (selectStrategy_safe V x Om20 f om h).2
  . rintro <<om, hom, hC>> hnone
    rw [selectStrategy_eq_none_iff] at hnone
    have hmem : om in Om20 cap compliantSet V x :=
      Finset.mem_inter.mpr <<hom, (mem_compliantSet V x om).mpr (
         hcomplete om hC)>>
    rw [hnone] at hmem
    exact absurd hmem (Finset.notMem_empty om)
```

Significance. The statement of Equation A.28 is encoded as a partial function with values in `Option`. Safety with respect to the semantic set C requires the soundness hypothesis of Theorem C.1.

20 Status Propagation (Section A.6, Equation A.18)

Statement. With $\kappa \in \{\text{candidate}, \text{confirmed}\}$ and the chain output carrying the signal s, $\kappa(\Phi_B(D)) = \kappa(s)$.

```
/-- Confirmation status of a signal, ordered candidate < confirmed. -/
inductive Status where
  | candidate
  | confirmed
  deriving DecidableEq

/-- Equation A.18: s = (S, ell, kappa). -/
structure Signal (iota Level : Type*) where
```

```
  members : Finset iota
  level : Level
  status : Status

/-- Output of the chain Phi_B(D) = (s, a*_B, om*). -/
structure ChainOutput (iota Level Recipe Strategy : Type*) where
  signal : Signal iota Level
  recipe : Recipe
  strategy : Strategy

/-- kappa of a chain output is the status of the signal that it carries.
    -/
def ChainOutput.status {iota Level Recipe Strategy : Type*}
    (o : ChainOutput iota Level Recipe Strategy) : Status :=
  o.signal.status

/-- T20. By the encoding chosen, the chain output carries the signal
    produced by Sigma, so status propagation holds by definition (`rfl`)
       . -/
theorem status_propagation {Corpus iota Level Recipe Strategy : Type*}
    (sigma : Corpus -> Signal iota Level) (prodOp : Signal iota Level ->
        Recipe)
    (gamma : Signal iota Level -> Recipe -> Strategy) (D : Corpus) :
    (ChainOutput.mk (sigma D) (prodOp (sigma D)) (gamma (sigma D) (
       prodOp (sigma D)))).status = (sigma D).status :=
  rfl
```

Significance. In the encoding chosen, the chain output stores the signal produced by the signal operator and its status is read from that field. The theorem holds by definition (`rfl`). It records that the encoding of the chain preserves the status; it does not derive the property from other assumptions.

21 Well-Typedness of the Composition (Section A.6, Equation A.30)

Statement. The signal operator returns a non-empty member set $S \subseteq F$; the product operator requires $S \neq \emptyset$ and returns a recipe whose ratios sum to 1; the strategy operator requires x in the domain of ψ. The composition $\Phi_B = \Gamma \circ \Pi_B \circ \Sigma$ is well defined.

```
/-- Output of Sigma: a non-empty member set contained in the family F.
    -/
structure SignalOver {iota : Type*} (F : Finset iota) where
  members : Finset iota
  subset : members subset F
  nonempty : members.Nonempty

/-- Output of Pi_B: a recipe whose eight ratios sum to one. -/
structure Recipe where
  ratios : Fin 8 -> R
  sum_one : sum n, ratios n = 1

/-- Context map psi : (recipe, observation) -> R10 and its image, the
    domain required by Gamma. -/
def ContextDomain {Obs : Type*} (psi : Recipe -> Obs -> (Fin 10 -> R)) :
    Type _ :=
  {x : Fin 10 -> R // exists a o, x = psi a o}
```

```
/-- Sigma requires a corpus and returns a SignalOver F and an
    observation;
    Pi_B requires a SignalOver F (non-empty by type) and returns a
       Recipe;
    Gamma requires a ContextDomain psi. The composition is well defined
       by
    construction. -/
noncomputable def chain {Corpus Obs Strategy iota : Type*} {F : Finset
    iota}
    (sigma : Corpus -> SignalOver F x Obs)
    (prodOp : SignalOver F -> Obs -> Recipe)
    (psi : Recipe -> Obs -> (Fin 10 -> R))
    (gamma : ContextDomain psi -> Strategy) (D : Corpus) :
    SignalOver F x Recipe x Strategy :=
  let so := sigma D
  let a := prodOp so.1 so.2
  (so.1, a, gamma <<psi a so.2, a, so.2, rfl>>)

/-- The three preconditions hold for the data flowing through the chain.
    -/
theorem chain_wellTyped {Corpus Obs Strategy iota : Type*} {F : Finset
    iota}
    (sigma : Corpus -> SignalOver F x Obs)
    (prodOp : SignalOver F -> Obs -> Recipe)
    (psi : Recipe -> Obs -> (Fin 10 -> R))
    (gamma : ContextDomain psi -> Strategy) (D : Corpus) :
    ((chain sigma prodOp psi gamma D).1.members.Nonempty /\ (chain sigma
        prodOp psi gamma D).1.members subset F) /\
    (sum n, (chain sigma prodOp psi gamma D).2.1.ratios n = 1) /\
    (exists a o, psi a o = psi (chain sigma prodOp psi gamma D).2.1 (
       sigma D).2) := by
  exact <<<<(sigma D).1.nonempty, (sigma D).1.subset>>, (prodOp (sigma D
     ).1 (sigma D).2).sum_one,
    <<_, _, rfl>>>>
```

Significance. Encoding: the preconditions are fields of structures. `SignalOver F` carries non-emptiness and inclusion in F. `Recipe` carries the sum of the eight ratios. `ContextDomain` is the subtype of $\mathbb{R}^{10}$ formed by the values of ψ. The function `chain` type-checks because each output satisfies the next precondition; the theorem restates the three invariants for the data flowing through the chain.

22 Sensitivity of CBR to λ with Different Times

Statement. For Equation A.10 with $\mathrm{StS}(s,t) = R_0(s)\exp(-\lambda t)/(t+t_0)$, $t_0 > 0$, $t_{\mathrm{int}}, t_{\mathrm{seq}} \geq 0$ and $R_0 > 0$ in both arms,

$$\ln \mathrm{CBR}(\lambda) = \ln \frac{R_{0,\mathrm{int}}}{R_{0,\mathrm{seq}}} + \ln \frac{t_{\mathrm{seq}} + t_0}{t_{\mathrm{int}} + t_0} + \lambda\,(t_{\mathrm{seq}} - t_{\mathrm{int}}). \tag{24}$$

Consequently CBR is strictly increasing in λ if $t_{\mathrm{int}} < t_{\mathrm{seq}}$, strictly decreasing if $t_{\mathrm{int}} > t_{\mathrm{seq}}$, and constant if $t_{\mathrm{int}} = t_{\mathrm{seq}}$.

```
theorem CBR_arms_closed_form
    (R0_int R0_seq lam t0 t_int t_seq : R)
    (hint : R0_int > 0) (hseq : R0_seq > 0)
    (ht0 : t0 > 0) (ht_int : t_int >= 0) (ht_seq : t_seq >= 0) :
```

```
    CBR_arms R0_int R0_seq lam t0 t_int t_seq =
    (R0_int / R0_seq * ((t_seq + t0) / (t_int + t0))) *
      Real.exp (lam * (t_seq - t_int)) := by
  have hi : t_int + t0 > 0 := StS_denominator_pos t0 t_int ht0 ht_int
  have hs : t_seq + t0 > 0 := StS_denominator_pos t0 t_seq ht0 ht_seq
  have e : Real.exp (-lam * t_int) =
      Real.exp (lam * (t_seq - t_int)) * Real.exp (-lam * t_seq) := by
    rw [<- Real.exp_add]
    congr 1
    ring
  unfold CBR_arms StS
  rw [e]
  have h1 : Real.exp (-lam * t_seq) <> 0 := (Real.exp_pos _).ne'
  field_simp

/-- T22. ln CBR(lam) = ln(R0_int / R0_seq) + ln((t_seq + t0)/(t_int + t0
    ))
    + lam (t_seq - t_int). -/
theorem CBR_log_sensitivity
    (R0_int R0_seq lam t0 t_int t_seq : R)
    (hint : R0_int > 0) (hseq : R0_seq > 0)
    (ht0 : t0 > 0) (ht_int : t_int >= 0) (ht_seq : t_seq >= 0) :
    Real.log (CBR_arms R0_int R0_seq lam t0 t_int t_seq) =
    Real.log (R0_int / R0_seq) + Real.log ((t_seq + t0) / (t_int + t0))
       +
      lam * (t_seq - t_int) := by
  have hi : t_int + t0 > 0 := StS_denominator_pos t0 t_int ht0 ht_int
  have hs : t_seq + t0 > 0 := StS_denominator_pos t0 t_seq ht0 ht_seq
  rw [CBR_arms_closed_form R0_int R0_seq lam t0 t_int t_seq hint hseq
     ht0 ht_int ht_seq,
    Real.log_mul (mul_pos (div_pos hint hseq) (div_pos hs hi)).ne' (Real
       .exp_pos _).ne',
    Real.log_mul (div_pos hint hseq).ne' (div_pos hs hi).ne', Real.
       log_exp]

/-- T22, consequences: strictly increasing in lam if t_int < t_seq,
    strictly decreasing if t_int > t_seq, constant if t_int = t_seq. -/
theorem CBR_lambda_monotonicity
    (R0_int R0_seq t0 t_int t_seq : R)
    (hint : R0_int > 0) (hseq : R0_seq > 0)
    (ht0 : t0 > 0) (ht_int : t_int >= 0) (ht_seq : t_seq >= 0) :
    (t_int < t_seq -> StrictMono (fun lam => CBR_arms R0_int R0_seq lam
       t0 t_int t_seq)) /\
    (t_seq < t_int -> StrictAnti (fun lam => CBR_arms R0_int R0_seq lam
       t0 t_int t_seq)) /\
    (t_int = t_seq -> forall lam1 lam2,
      CBR_arms R0_int R0_seq lam1 t0 t_int t_seq =
      CBR_arms R0_int R0_seq lam2 t0 t_int t_seq) := by
  have hi : t_int + t0 > 0 := StS_denominator_pos t0 t_int ht0 ht_int
  have hs : t_seq + t0 > 0 := StS_denominator_pos t0 t_seq ht0 ht_seq
  have hK : 0 < R0_int / R0_seq * ((t_seq + t0) / (t_int + t0)) :=
    mul_pos (div_pos hint hseq) (div_pos hs hi)
  have hform := fun lam => CBR_arms_closed_form R0_int R0_seq lam t0
     t_int t_seq
    hint hseq ht0 ht_int ht_seq
  refine <<?_, ?_, ?_>>
  . intro hlt lam1 lam2 h12
    simp only [hform]
```

```
    apply mul_lt_mul_of_pos_left _ hK
    apply Real.exp_lt_exp.mpr
    nlinarith
  . intro hlt lam1 lam2 h12
    simp only [hform]
    apply mul_lt_mul_of_pos_left _ hK
    apply Real.exp_lt_exp.mpr
    nlinarith
  . intro heq lam1 lam2
    rw [hform, hform, heq]
    simp

/-- Consistency with the existing theorem `
    CBR_lambda_invariant_equal_time`:
    the equal-time case of T22 gives the same conclusion. -/
theorem CBR_lambda_invariant_from_sensitivity
    (R_base R_prod R_mark R_coup t0 t lam1 lam2 : R)
    (ht0 : t0 > 0) (ht : t >= 0)
    (hR_base_pos : R_base + R_prod + R_mark > 0)
    (hcoup : R_coup > 0) :
    CBR R_base R_prod R_mark R_coup lam1 t0 t t =
    CBR R_base R_prod R_mark R_coup lam2 t0 t t := by
  have hint : R_base + R_prod + R_mark + R_coup > 0 := by linarith
  rw [CBR_eq_CBR_arms, CBR_eq_CBR_arms]
  exact (CBR_lambda_monotonicity _ _ t0 t t hint hR_base_pos ht0 ht ht)
     .2.2
    rfl lam1 lam2
```

Significance. This closes the limitation declared in Section A.1 on the sensitivity of CBR to λ under unequal deployment times. The equal-time case reproduces Theorem 13 (`CBR_lambda_invariant_from_se` derives its conclusion from the new formula).

23 Sign of the Speed Term (Equation A.29)

Statement. With $\Delta t_{\text{int}} = 4.0 - \min(1.5,\, b + 0.2 \cdot \mathbb{1}[\rho])$, $b \geq 0$, $\mathbb{1}[\rho] \in \{0, 1\}$ and $\Delta t_{\text{seq}} = 9.0$ (months): $2.5 \leq \Delta t_{\text{int}} \leq 4.0 < \Delta t_{\text{seq}}$. For $\lambda > 0$, $\lambda(\Delta t_{\text{seq}} - \Delta t_{\text{int}}) > 0$ and CBR $> R/p$.

```
/-- Dt_int of Equation A.29, in months. -/
noncomputable def timeToShelfInt (b : R) (rho : Prop) [Decidable rho] :
    R :=
  4.0 - min 1.5 (b + 0.2 * (if rho then 1 else 0))

/-- Dt_seq of Equation A.29, in months. -/
def timeToShelfSeq : R := 9.0

/-- T23. With b >= 0: 2.5 <= Dt_int <= 4.0 < Dt_seq. -/
theorem timeToShelf_bounds (b : R) (rho : Prop) [Decidable rho] (hb : 0
    <= b) :
    2.5 <= timeToShelfInt b rho /\ timeToShelfInt b rho <= 4.0 /\ 4.0 <
       timeToShelfSeq := by
  unfold timeToShelfInt timeToShelfSeq
  have h0 : (0 : R) <= 0.2 * (if rho then (1 : R) else 0) := by
    split_ifs <;> norm_num
  refine <<?_, ?_, by norm_num>>
  . have := min_le_left (1.5 : R) (b + 0.2 * (if rho then 1 else 0))
    linarith
```

```
  . have := le_min (by norm_num : (0 : R) <= 1.5) (by linarith : (0 : R)
      <= b + 0.2 * (if rho then 1 else 0))
    linarith

/-- The lower bound 2.5 <= Dt_int holds without any hypothesis on b. -/
theorem timeToShelfInt_lower (b : R) (rho : Prop) [Decidable rho] :
    2.5 <= timeToShelfInt b rho := by
  unfold timeToShelfInt
  have := min_le_left (1.5 : R) (b + 0.2 * (if rho then 1 else 0))
  linarith

/-- T23, consequences: for lam > 0, lam (Dt_seq - Dt_int) > 0 and
    CBR > R/p. The factor c > 0 converts months to the unit of 1/lam
    (Section A.4 uses c = 365.25/(12 x 7) weeks per month). -/
theorem speed_term_positive (b : R) (rho : Prop) [Decidable rho] (hb : 0
    <= b)
    (lam c R p : R) (hlam : lam > 0) (hc : c > 0) (hR : R > 0) (hp : p >
        0) :
    lam * (timeToShelfSeq - timeToShelfInt b rho) > 0 /\
    lam * (c * timeToShelfSeq - c * timeToShelfInt b rho) > 0 /\
    CBR_impl R p lam (c * timeToShelfInt b rho) (c * timeToShelfSeq) > R
        / p := by
  obtain <<_, h2, h3>> := timeToShelf_bounds b rho hb
  have hgap : timeToShelfSeq - timeToShelfInt b rho > 0 := by linarith
  have hgapc : c * timeToShelfSeq - c * timeToShelfInt b rho > 0 := by
    rw [<- mul_sub]; exact mul_pos hc hgap
  refine <<mul_pos hlam hgap, mul_pos hlam hgapc, ?_>>
  unfold CBR_impl
  have hexp : 1 < Real.exp (-lam * (c * timeToShelfInt b rho - c *
     timeToShelfSeq)) := by
    rw [<- Real.exp_zero]
    apply Real.exp_lt_exp.mpr
    nlinarith
  have hq : 0 < R / p := div_pos hR hp
  calc R / p = R / p * 1 := by ring
    _ < R / p * Real.exp (-lam * (c * timeToShelfInt b rho - c *
       timeToShelfSeq)) :=
      mul_lt_mul_of_pos_left hexp hq

end FMCG40
```

Significance. The lower bound $2.5 \leq \Delta t_{\text{int}}$ holds for every b. The upper bound requires $b + 0.2 \cdot \mathbb{1}[\rho] \geq 0$; the hypothesis $b \geq 0$ is stated explicitly. A factor $c > 0$ converts months to the unit of $1/\lambda$ (weeks), and the signs are unchanged by c.

Summary

Scope Limitations, Stated Honestly

- This verification establishes internal mathematical consistency of the model as defined — it does not constitute empirical validation against real-world FMCG data.
- The sensitivity of CBR to λ under *unequal* deployment times ($t_{\text{int}} \neq t_{\text{seq}}$) is characterised by Theorem 22; Theorem 13 covers the equal-time case.
- Encoders, cosine similarities, the neural policy, the gradient boosting models and the ASP verifier are abstract functions with explicit hypotheses. The theorems hold for any

#	Theorem	Category
1	StS_denominator_pos	Well-definedness
2	StS_well_defined	Well-definedness
3	StS_max_value	StS properties
4	MarketResponse_pos	Positivity
5	TemporalDecay_bounds	Bounds
6	trend_half_life	Decay properties
7	decay_uniqueness_given_derivative_at_zero	Decay properties
8	eta_coup_bounds	CBR components
9	decay_rate_estimator_well_defined	Parameter estimation
10	StS_strictly_decreasing	Monotonicity
11	StS_max_at_zero	Monotonicity
12	CBR_geq_one	CBR main result
13	CBR_lambda_invariant_equal_time	CBR sensitivity
14	CBR_eq_one_iff_no_coupling	CBR special case
15	CBR_log_decomposition	Chain, Eq. A.33
16	CBR_gt_one_iff, CBR_condition10_strict	Chain, exact condition
17	coverageReward_bounds, consumerReward_bounds	Chain, boundedness
18	bestWithin_mono	Chain, budget monotonicity
19	selection_mem_compliant, selection_theorem_C1	Chain, safety
20	status_propagation	Chain, status
21	chain_wellTyped	Chain, well-typedness
22	CBR_log_sensitivity, CBR_lambda_monotonicity	CBR sensitivity
23	timeToShelf_bounds, speed_term_positive	Chain, speed term

Table 1: Complete list of the numbered machine-checked theorems.

implementation that satisfies those hypotheses, and they say nothing about the quality of a particular implementation.

- Theorem 7's uniqueness result is proven under an explicit differentiability hypothesis, not derived from continuity and the memoryless functional equation alone; the fully general classical characterisation remains unformalised.